HelixVPN — Cross-Document Synthesis (evidence base for the master spec)

HelixVPN — Cross-Document Synthesis (evidence base for the master spec)

> Distilled from full digests of all 16 source research docs in `docs/research/mvp/` (11 LLM analyses 00/01/02/03/05/06/07/08/09/10/11 + the 5 `04_VPN_CLD` refined docs). This is the cross-cutting synthesis; spec writers also read the original source docs for depth. Cite sources by id, e.g. [04_ARCH §3], [02_QWN], [11_MST].

1. What HelixVPN is (the settled product floor)

A **self-hostable overlay network with a privacy-VPN front end**. Pitch [04_ARCH §1]: "Cloudflare Tunnel + WARP, rebuilt as Tailscale-style coordination, with Mullvad's obfuscation stack, fully self-hostable, on one shared codebase."

Founding constraint [00, 05_YBO]: give remote users full access to one **or many** internal/home/lab networks **without any inbound port-forward** — internal hosts dial *outbound* to a public gateway (reverse tunnel); the gateway relays/routes. Differentiator: **1 user → N joined private networks** (multi-network bidirectional gateway), with per-user ACL routing.

Three roles: Connector (network-side agent, outbound-only, advertises CIDRs) ⇄ **Gateway** (public VPS: control + data plane, routes/polices) ⇄ **Client** (Mullvad-style end-user app; gets overlay IP; reaches authorized networks or uses gateway as plain privacy exit). **Three apps:** Access (end user), Connector (network operator), Console (admin). [05_YBO, 04_ARCH §1, consensus across all 10 analyses]

2. Settled stack floor (near-unanimous consensus)

- **Control-plane backend (mandated [05_YBO], confirmed by all):** **Go + Gin + PostgreSQL
  - Redis + Podman (rootless)**. Postgres = source of truth (multi-tenant, RLS); Redis = ephemeral presence + event bus. Matches constitution §11.4.76 (containers submodule) + §11.4.161 (rootless).
- **Crypto core: WireGuard** (Noise IK, Curve25519, ChaCha20-Poly1305) is the cryptographic core everywhere; **obfuscation/transport is pluggable beneath WG** — never fork WG crypto [04_ARCH §2/§3].
- **Architecture:** reverse-tunnel hub-and-spoke; outbound-only edges; push-don't-poll (event-driven desired-state "network map", Tailscale MapResponse-style); control/data plane strictly separated (Go never in packet path; fail-static).
- **Stance:** self-hosted / home-lab first (no-logs because you own it), same code can serve managed later. Licensing (source-available + commercial) is an open decision.
- **UI: Flutter** dominant choice for cross-platform reach + a shared native core.
- **Platforms (all required [05_YBO]):** iOS, Android, Aurora OS, HarmonyOS NEXT, Windows, Linux, macOS, Web (responsive; Web = admin/proxy only — no system TUN in a browser).
- **Real-time:** event-driven config push (Redis Streams/pub-sub → WS/SSE to apps; server-streaming RPC to agents). Obfuscation treated as **mandatory**, "especially QUIC".

3. THE KEY DECISIONS / DIVERGENCES (must be presented explicitly, not silently resolved)

The `04_VPN_CLD` refined docs pivoted to one camp; the broader 10-LLM consensus differs. The master spec MUST surface these as decisions-with-recommendations, analyzed from all angles.

#	Decision	Camp A	Camp B	Notes / recommendation surface
D1	**Primary obfuscating transport**	**MASQUE/QUIC** = WG-over-HTTP-3 (RFC 9298/9297/9221), Mullvad's actual mechanism	**Hysteria2 + Salamander** (QUIC+obfs, turnkey) primary, WG fallback [plurality of 10 analyses]	CLD argues MASQUE = true Mullvad parity + single Rust impl; consensus argues Hysteria2 ships faster. Spec must

#	Decision	Camp A	Camp B	Notes / recommendation surface
		[04_ARCH §3.3, 04_P0]		compare; note "Mullvad QUIC ≠ separate protocol, it IS WG-over-MASQUE" [04_ARCH §1].
D2	**Shared client-core language**	**Rust** core + Flutter UI [04_*, QWN, ZAI, CPL, KMI, MST — plurality]	**Go** core + Flutter UI (reuses Hysteria2's Go) [DSK, GMI]	Rust wins on iOS memory ceiling + WASM; Go simpler + reuses Hysteria2. CLD = Rust.
D3	**Event bus**	**Redis Streams** (MVP) → NATS JetStream (scale) [04_P1, QWN, MST]	**NATS JetStream** from start [DSK, KMI]	CLD: Redis MVP, NATS Phase 2.
D4	**IP-subnet collision** across N joined RFC1918 nets	**IPv6 ULA /48 per tenant + Tailscale 4via6** [04_ARCH §3.4]	**CGNAT 100.64/10 1:1 per network** [GMI, KMI]	Only GMI/KMI/CLD actually solve it — a v1 must-decide.
D5	**Gateway edge language** (MASQUE termination)	**Rust** (quinn+h3, shares helix-transport byte-for-byte) [04_P0 G4]	**Go** (quic-go + masque-go turnkey)	Phase-0 gate G4 decides by benchmark.
D6	**Transport topology**	single protocol end-to-end	**asymmetric per-leg**: Hysteria2/QUIC user?gateway, WireGuard gateway?networks [11_MST — distinctive]	MST's best-fit-per-leg is elegant; worth surfacing.
D7	**MVP ambition**	lean tunnel-first (CLI between 2 hosts → API → apps) [QWN, DSK, 04_P0]	full ecosystem/"Connectivity-OS" from v1 [KMI, ZAI]	Recommend lean Phase-0 spike then MVP.

4. Phased structure (the CLD roadmap — the spine for phases→tasks→subtasks)

- **Phase 0 — Spike (~3-4 wk, throwaway bodies on production interfaces).** Exit gates: G1 plain-UDP WG client→gw→connector LAN (⩾80% bare-link); G2 MASQUE/QUIC through a DPI UDP block (⩾50% of plain); **G3 iOS NEPacketTunnelProvider Rust core under memory ceiling ⩾30% headroom (make-or-break)**; G4 Go-vs-Rust edge benchmark decision; G5 flutter_rust_bridge FFI drives core from Dart; G6 push-based reconcile from a static map. Milestones S0–S8. Surviving interfaces: `Transport` trait, `helix-wg` boringtun wrapper, orchestrator + status enum, FFI surface. Test rig: Linux netns + nftables DPI sim + tc netem. [04_P0]
- **Phase 1 — MVP (self-hostable).** Go modular monolith (identity/ registry/ipam/pki/policy/ coordinator/events/telemetry/api/store); Postgres + RLS data model (no connection/traffic tables — CI-lint enforced); protobuf `Coordinator` over Connect with server-streaming **`WatchNetworkMap`** (snapshot+deltas, peers already policy-filtered, need-to-know); coordinator (in-mem topology graph, minimal deltas, **p99 <1s** convergence); Redis Streams backbone; identity (OIDC + anonymous device tokens) + enrollment (device-gen WG keypair, private key never leaves) + short-lived mTLS device cert + revoke <1s; policy compiler (Tailscale-ACL-flavored, default-deny, fail-closed, → AllowedIPs + nftables/eBPF verdict maps); Gin REST + WS/SSE; `helixvpnctl` (Cobra) + Podman quadlets; hub-and-spoke data path. **MVP DoD = 8 acceptance criteria** (self-host from zero; enroll connector+client; reach authorized LAN host + deny unauthorized; auto-escalate to MASQUE when WG blocked; policy edit <1s no restart; revoke <1s; kill-switch+DNS-leak; no durable conn log; 3 apps drive it). [04_P1]
- **Phase 2 — Parity + Reach.** Full transport set (+Shadowsocks, UDP-over-TCP, hardened LWO, auto-ladder + per-network memory + regional priors); **DAITA via maybenot**; **direct P2P + NAT traversal** (STUN-like discovery, hole punching, DERP-style helix-relay fallback); **multi-hop** nested WG; **post-quantum** handshake (ML-KEM/FIPS-203 PSK, hybrid-never-PQ-only, Rosenpass alt); desktop apps (Windows wireguard-nt+service, macOS NE); policy-as-code/GitOps; HA + multi-region (stateless coordinators, Patroni PG, NATS JetStream). [04_P2]
- **Phase 3 — Extended reach.** HarmonyOS NEXT + Aurora OS builds (real native tunnel-shim work — biggest platform risk), WASM browser MASQUE proxy, billing-optional multi-tenant, third-party audit + reproducible builds.

5. Client architecture (shared-codebase strategy) [04_ARCH §5, 04_UI]

- **`helix-core` (Rust):** WG control (boringtun/kernel), `helix-transport` obfuscation crate, kill-switch, DNS, network-map reconciliation, FFI surface. → flutter_rust_bridge v2 (Dart), UniFFI (native shims). Same precedent as Mullvad daemon / Cloudflare WARP.
- **`helix-ui` (Flutter):** all screens + `helix_design` system (Material 3 + brand tokens, connection-state palette, signature components: ConnectButton/StatusChip/ExitPicker/ ShieldIndicator/ AdaptiveScaffold), Riverpod state = pure function of core status stream. Three flavors from one tree via `runHelixApp(flavor, home, capabilities)`: Access / Connector / Console (Console = only Web build, no core_ffi). Melos monorepo.
- **Per-platform `TunnelPlatform` shim (the only platform-specific code):** iOS/macOS NEPacketTunnelProvider (Swift); Android VpnService+JNI (Kotlin); Windows wireguard-nt/wintun
  - privileged service (named-pipe IPC); Linux kernel WG/tun (Rust); HarmonyOS Network Kit ability (ArkTS→NAPI→.so); Aurora Qt/C++ + tun; Web none. Budget: iOS NE ~15 MB historical ceiling (measure on device) — the reason core is Rust not Go.

6. Repo layout (decoupling target) [04_ARCH §11]

```
helixvpn/  (root)
├── helix-core/   # Rust: crates/helix-transport, helix-wg, helix-ffi   → reusable
├── helix-edge/   # Rust: gateway data-plane edge (uses helix-transport) → reusable
├── helix-go/     # Go control plane (modular monolith)                  → reusable
├── helix-proto/  # Protobuf + OpenAPI → generated Dart/Go/Rust clients  → reusable
├── helix-ui/     # Flutter: design system + screens + 3 flavors         → reusable
├── shims/        # apple/ android/ windows/ linux/ harmonyos/ aurora/
├── deploy/       # Podman quadlets, Terraform, Grafana-as-code, helixvpnctl
└── (Helix ecosystem submodules already incorporated under submodules/)
```

Three reuse pillars: Rust core (client+connector+edge), Flutter UI (all apps), schema-generated clients (all langs).

7. Security / privacy invariants (non-negotiable) [04_ARCH §7, 04_P1 §11.4]

Zero-trust default-deny; device private keys never leave device; peers delivered already-policy-filtered (need-to-know); short-lived mTLS device certs + WG Noise data channel; rootless Podman + read-only rootfs + seccomp + NET_ADMIN-only + no-SSH on edge; **no-logging as code** — only aggregate counters; **CI schema-lint fails build if any durable connection/traffic/packet table appears**; control actions (not traffic) audited; kill-switch

- DNS-leak protection driven by core state machine; PQ PSK optional.

8. Helix-ecosystem integration the source docs MISS (must be designed in)

The research predates the just-incorporated `submodules/`: **containers** (vasic-digital) is the §11.4.76 mandated container orchestration layer for deploy + on-demand integration-test infra; **helix_qa** + **challenges** are the anti-bluff QA/Challenge layer (§11.4.27/§11.4.5/.69/.107); **docs_chain** mechanizes the spec-doc + workable-items sync (§11.4.106); **security** = security tooling (§7); **vision_engine** = video-evidence QA (§11.4.107/.158); **llm_***/panoptic where relevant. The master spec must wire each in or mark not-applicable with reason.

9. Constitution bindings that shape THIS deliverable

§11.4.93/.95 (workable-items SQLite DB, git-tracked, single source of truth — phases/tasks/ subtasks become items), §11.4.106 (docs_chain sync engine), §11.4.65/.153 (HTML+PDF[+DOCX] exports for every doc), §11.4.28/.29/.74 (decoupled reusable components → own vasic-digital repos, snake_case, flat submodules), §11.4.36 (upstreams/ + install_upstreams per repo), §11.4.151/.155 (release/recording prefixes), §11.4.156 (NO active CI), §11.4.113 (no force-push), §11.4.66/.101 (decision discipline), §11.4.5/.69/.107 (captured-evidence / anti-bluff).